AI-Based Intrusion Detection System

Literature Survey, Software Requirements Specification (SRS) and System Analysis

Prepared by: [Your Name] | [Course], [College / University]

1. Introduction

The AI-Based Intrusion Detection System is a machine learning-based application that detects suspicious network activity and potential cyberattacks. Network traffic is captured with Wireshark, converted into flow-based features, and classified by a trained model (such as Random Forest, Decision Tree or XGBoost) as normal or as a specific attack type. Detected attacks generate alerts, are saved in activity logs, and are displayed on a dashboard.

Main objective: To detect and classify malicious network traffic automatically using machine learning, so that threats can be identified quickly.

Technologies: Python, Scikit-learn, Wireshark, CICFlowMeter, Pandas, Streamlit.

2. Literature Survey

Intrusion detection has been studied through rule-based, anomaly-based and machine learning approaches. The key works relevant to this project are summarised below.

Author(s), Year	Work	Approach	Relevance
Denning (1987)	An Intrusion-Detection Model	Models normal behavior and flags deviations as possible intrusions	Foundation of anomaly-based detection
Sommer, Paxson (2010)	Outside the Closed World: On Using Machine Learning for Network Intrusion Detection	Analysis of why ML is hard to apply to intrusion detection in real networks	Highlights false positives and the lab-to-real gap
Buczak, Guven (2016)	A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection	Survey of decision trees, SVM, k-NN, neural networks and clustering for IDS	Compares ML methods for this problem
Sharafaldin, Lashkari, Ghorbani (2018)	Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization	CIC-IDS2017: labeled flow data with DoS, DDoS, brute force, web attacks and port scans	Dataset used for training and evaluation
Breiman (2001); Chen, Guestrin (2016)	Random Forests; XGBoost	Tree ensembles and gradient boosting for tabular data	Strong, fast baseline classifiers
Liu, Ting, Zhou (2008)	Isolation Forest	Unsupervised anomaly detection by random partitioning	Option for detecting unseen attacks
Engelen, Rimmer, Joosen (2021)	Troubleshooting an Intrusion Detection Dataset: The CICIDS2017 Case Study	Finds labeling and flow-construction problems in CIC-IDS2017	Shows need for careful data cleaning
Vinayakumar et al. (2019)	Deep Learning Approach for Intelligent Intrusion Detection System	Deep neural networks applied to intrusion detection datasets	Future scope beyond classical ML

Findings and research gap. Tree-based models (Random Forest, XGBoost) on flow features give strong, fast and fairly interpretable baselines, while deep models need more data and compute. Many existing works stop at offline accuracy on benchmark data and do not provide a complete usable pipeline. This project addresses that gap with an end-to-end system covering capture, feature extraction, classification, alerting, logging and a dashboard, tested on lab-generated traffic as well as the benchmark dataset.

Note: Please verify all references on Google Scholar before submitting.

3. Software Requirements Specification (SRS)

3.1 Purpose and Scope

The system lets a user check network traffic and identify likely attacks. It is a detection tool only; it alerts the user but does not block traffic automatically. It gives a prediction based on flow patterns, so it works best on attack types seen in training.

3.2 Overall Description

Users: Network administrators, security analysts, students and evaluators.
Operating environment: Python 3.x, Scikit-learn, Wireshark, CICFlowMeter, Streamlit, web browser.
Assumptions: Live traffic is converted with the same features used in training, and testing is done only on networks the developer owns.

3.3 Functional Requirements

FR1: The user can upload a flow-based CSV file or a PCAP-derived file.
FR2: The system pre-processes the data (removes duplicates, handles missing and infinite values, scales features, handles class imbalance).
FR3: The system trains and compares at least three models and reports precision, recall, F1-score and a confusion matrix.
FR4: The trained model classifies each flow as Benign or an attack type.
FR5: The system generates an alert with timestamp, source IP, destination IP, attack type and confidence score.
FR6: The system stores all detections and alerts in a log file or database.
FR7: The dashboard shows alerts, attack counts, a timeline and top source IPs.
FR8: The system rejects invalid files with a clear error message.

3.4 Non-Functional Requirements

Performance: Classify a batch of flows within a few seconds on a standard laptop.
Accuracy: High recall on attack classes, reported per class with precision, recall and F1-score; target values fixed after baseline testing.
Usability: Simple dashboard that needs no ML knowledge.
Reliability and portability: Model saved with joblib; runs on Windows or Linux with Python.
Security and privacy: Logs are stored locally; testing only on authorised networks.

3.5 Hardware and Software Requirements

Minimum 8 GB RAM (16 GB recommended for the full dataset), i5 or equivalent processor, 10 GB free disk; Python, Scikit-learn, Pandas, NumPy, XGBoost, Wireshark, CICFlowMeter, and Streamlit or Flask for the dashboard.

4. System Analysis

4.1 Existing System

Networks are currently monitored with rule-based tools and manual log review. Signature rules only catch known attacks, and analysts cannot manually read millions of flows, so detection is slow and threats can go unnoticed.

4.2 Proposed System

An automated classifier trained on a labeled flow dataset (CIC-IDS2017). It pre-processes the data, classifies each flow, raises alerts, keeps logs and displays results on a dashboard, giving quick visibility of attacks.

4.3 Feasibility Study

Technical: Feasible, since Python, Scikit-learn and Wireshark are free and well documented.
Economic: Very low cost; uses open-source tools and a public dataset.
Operational: Easy to use through a simple dashboard; no special training needed.

4.4 System Workflow

1. **Data collection:** CIC-IDS2017 CSV for training; own lab traffic captured with Wireshark for live testing.
2. **Pre-processing:** Cleaning, removing duplicates and infinite values, scaling, handling class imbalance.
3. **Feature extraction:** Flow features (duration, packets, bytes, rates, flag counts) via CICFlowMeter.
4. **Model training:** Train Random Forest, Decision Tree and XGBoost; split data 80/20.
5. **Evaluation:** Precision, recall, F1-score and confusion matrix; select the best model.
6. **Deployment:** Traffic is classified, alerts are logged and the dashboard displays the results.

4.5 Limitations and Future Scope

The system detects only attack types seen in training and cannot inspect encrypted payloads, so it may miss new attacks and can produce false alarms. Future work includes anomaly detection with Isolation Forest, deep models (CNN, LSTM), log-based detection, email alerts and automatic blocking through firewall rules.

5. Conclusion

The proposed system offers a fast, low-cost and practical way to detect network attacks using machine learning, supported by established research, a realistic dataset and a clear set of requirements.